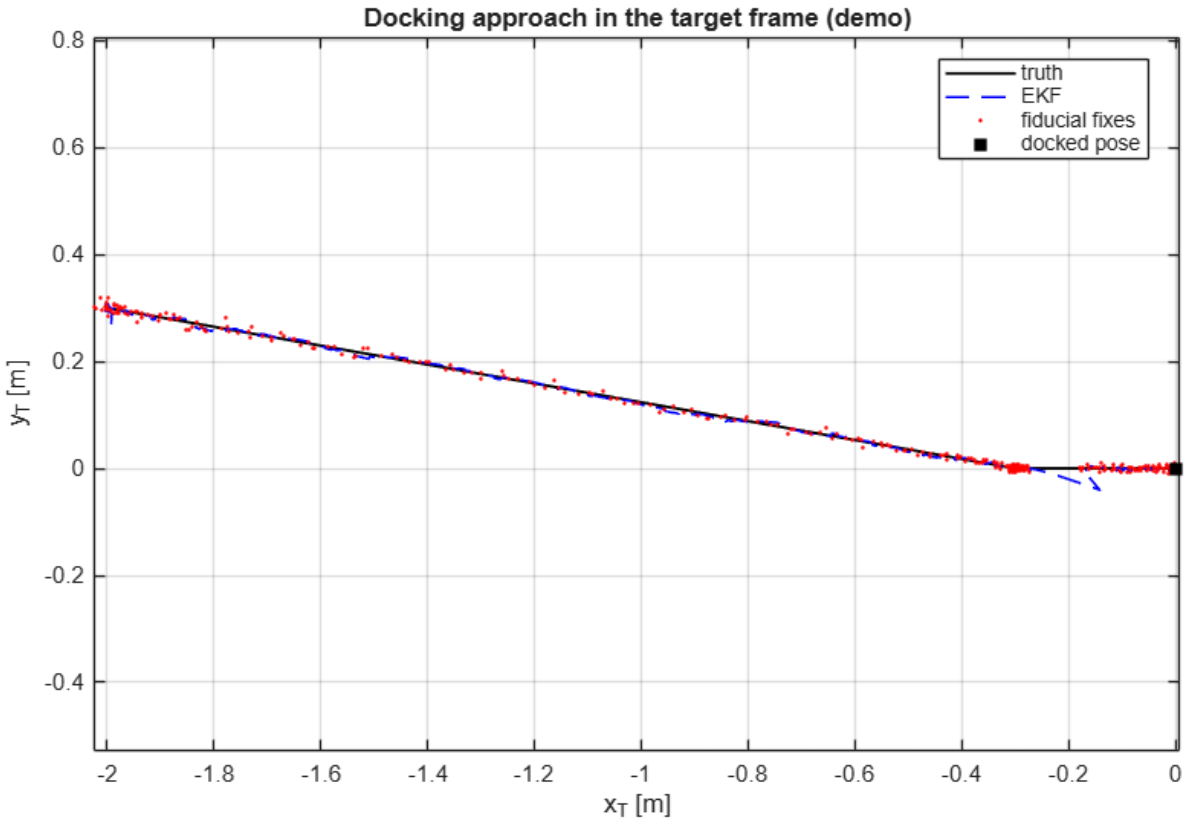
Figure catalogue: docking estimator and controller

Every figure exists for both instances, `demo` (tabletop VEX robot) and `fullscale` (600 kg barrier robot, simulation only), generated from the implemented MATLAB (`setup_paths`, then `make_ekf_figures`, `make_mpc_figures`, `make_report_figures`). Conventions: target frame with the origin at the docked chassis position and x into the target; black = truth, blue dashed = EKF estimate, green dashed = internal target or engagement band, red dashed = limit or tolerance, magenta dotted = supervisor stage switch. The numbers quoted are in `report_summary_<variant>.txt` and the output of `run_all_tests`. `README.md` in this folder lists which figure goes where in the report.

1 Estimator figures

Common setup: analytic truth trajectory (quintic staged approach from $(-2\ \text{m}, 0.3\ \text{m}, 15°)$ to the standoff and on to the dock), simulated 200 Hz IMU with white noise and residual bias, 15 Hz fiducial fixes with range-dependent noise and a 50 ms delay, a 2 s marker dropout at $t = 18$ to 20 s, handover initialisation (truth plus a draw from $\mathbf{P}_0$).

`ekf_demo_plane.png`

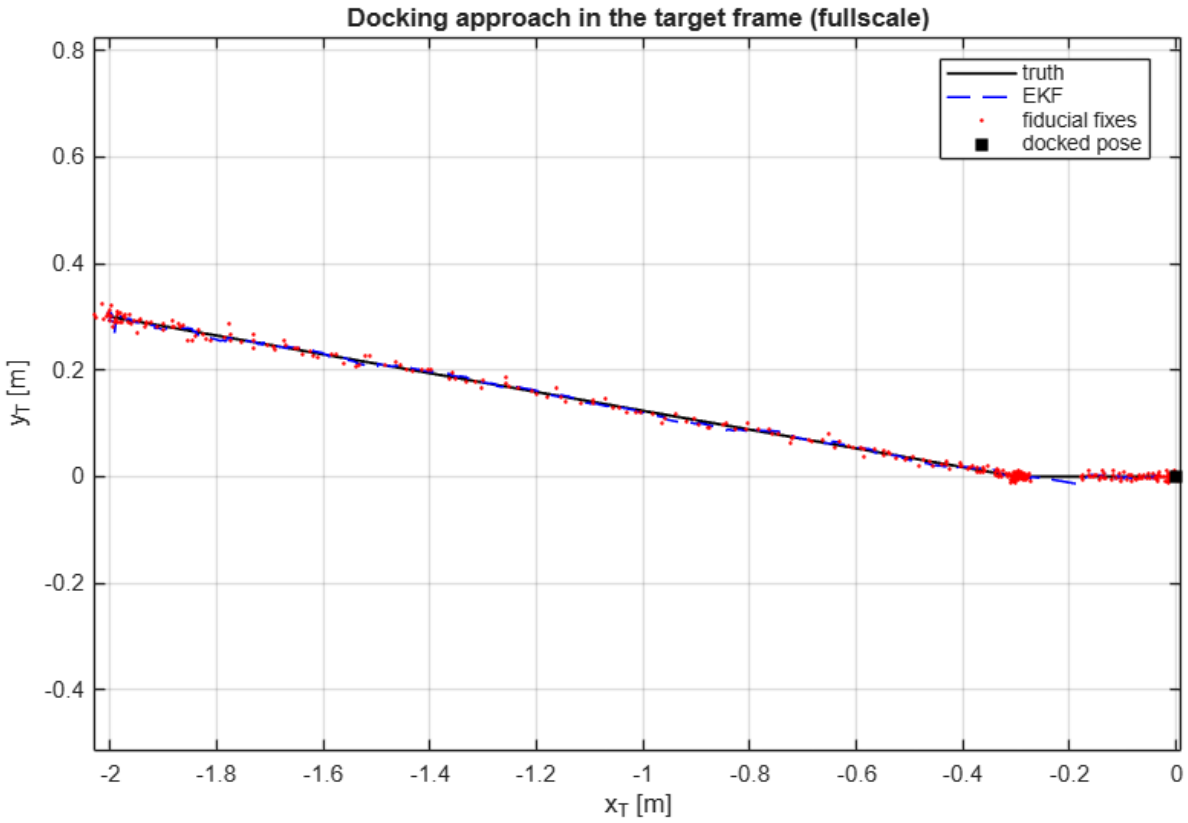


What it shows. Plane view in the target frame: truth path, EKF estimate, the fiducial fixes as points, the docked pose.

How it was made. `make_ekf_figures`, demo parameters.

What it supports. Context figure: the estimate rides on the truth, the fixes scatter with the range-dependent noise, the estimate coasts through the dropout.

`ekf_fullscale_plane.png`

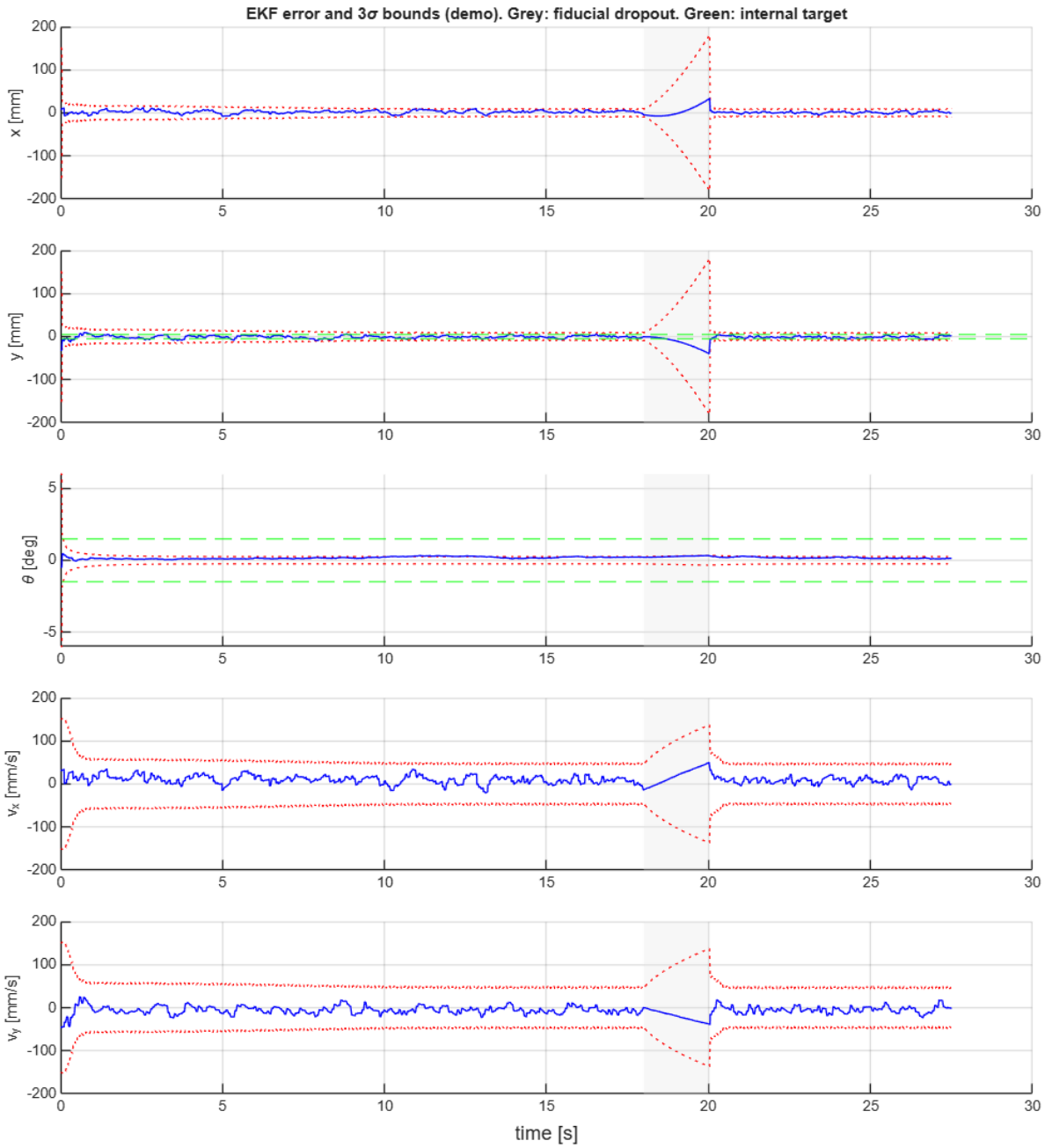


What it shows. As above, full-scale parameters (GQ7-class IMU, 5 mm fiducial noise at 1 m).

How it was made. `make_ekf_figures`, full scale.

What it supports. Same role for the full-scale instance.

`ekf_demo_errors.png`

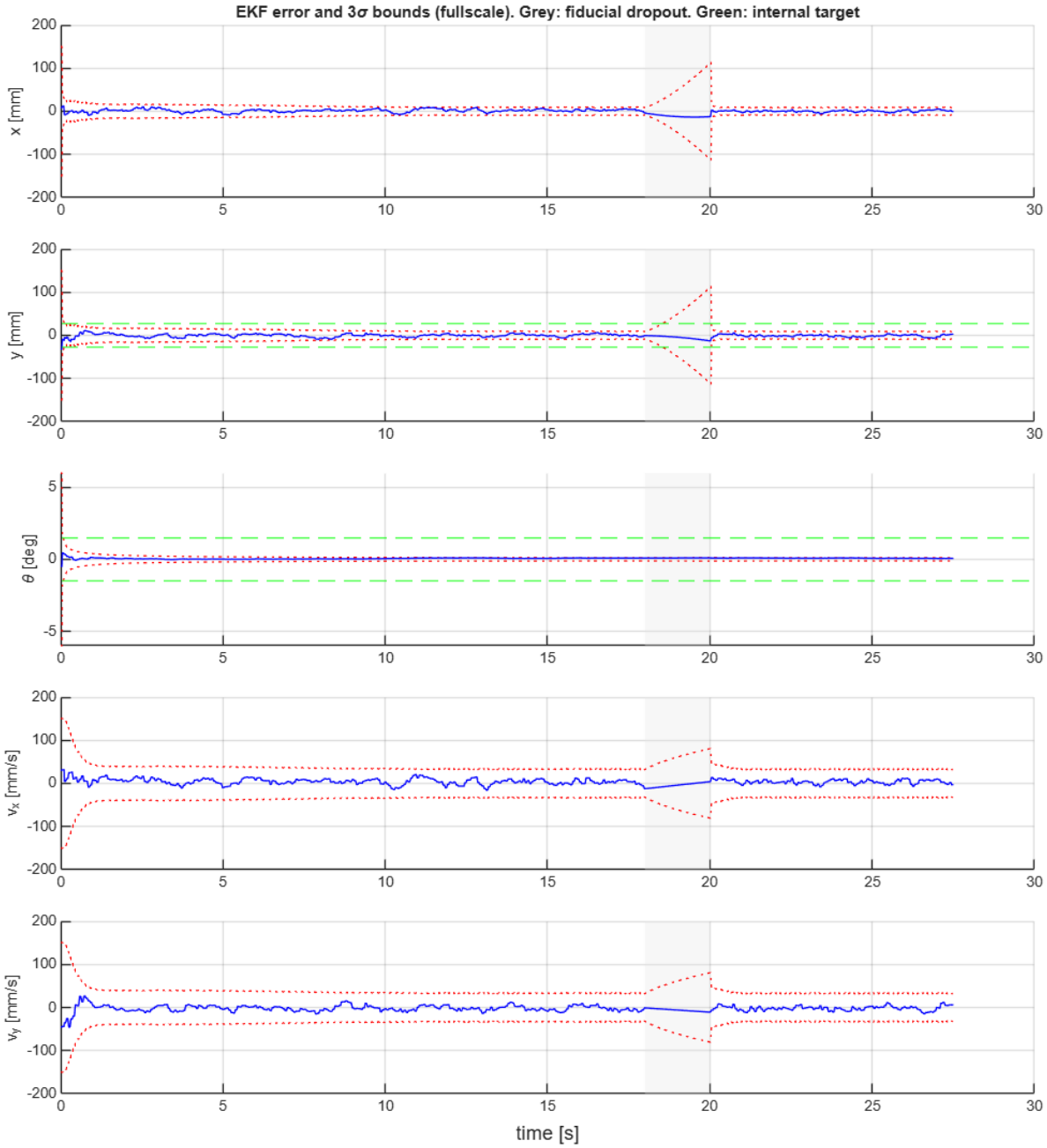


What it shows. Estimation error of x, y, θ, v_x, v_y against time with the filter's own $\pm 3\sigma$ bounds; grey band = dropout; green = internal target.

How it was made. Same run as the plane view.

What it supports. Consistency by inspection (errors inside the bounds about 99 % of the time), error growth during the dropout and re-convergence after it, and the 10 to 15 mm/s velocity-error level the MPC inherits through its initial increment.

`ekf_fullscale_errors.png`

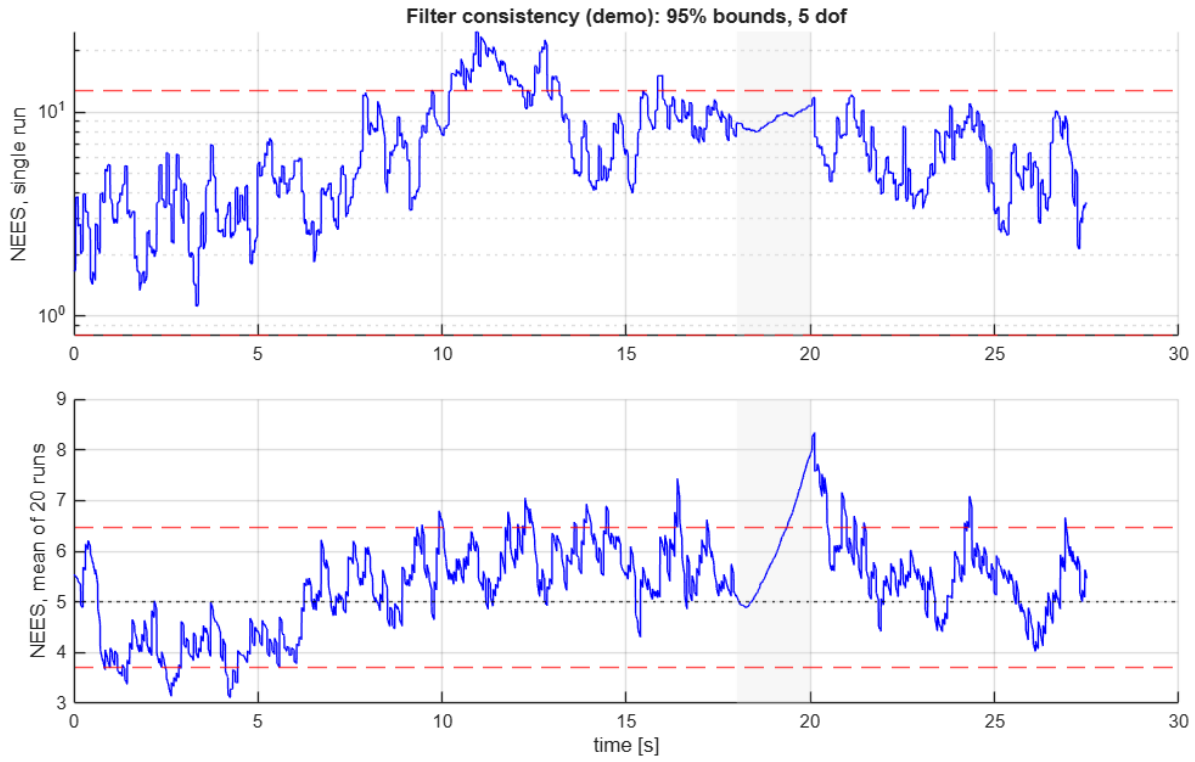


What it shows. As above, full scale.

How it was made. `make_ekf_figures`, full scale.

What it supports. Same role; note the smaller velocity error from the better IMU.

`ekf_demo_nees.png`

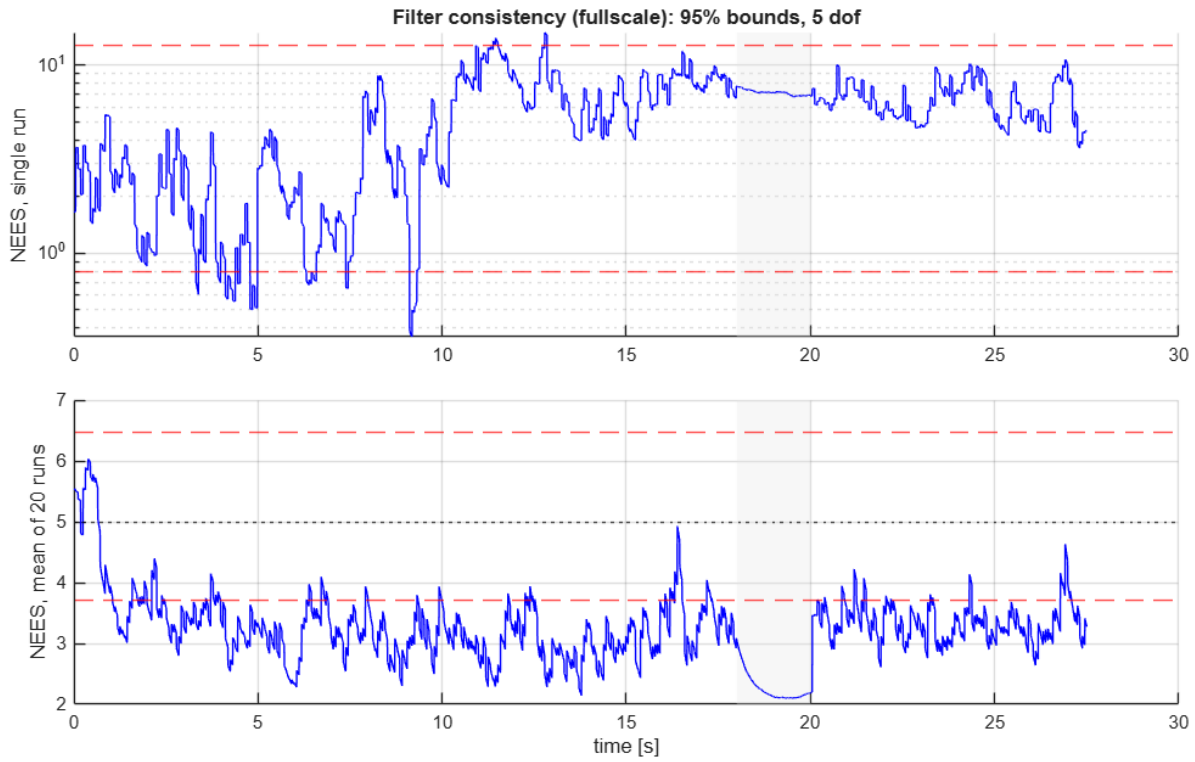


What it shows. Top: single-run NEES against time with the 95 % chi-square bounds (5 dof). Bottom: mean NEES of 20 Monte Carlo runs with the band.

How it was made. 20 runs with different noise seeds, same trajectory.

What it supports. The course's consistency test, used to choose the velocity process noise q_v: demo mean 5.32, inside the band.

`ekf_fullscale_nees.png`

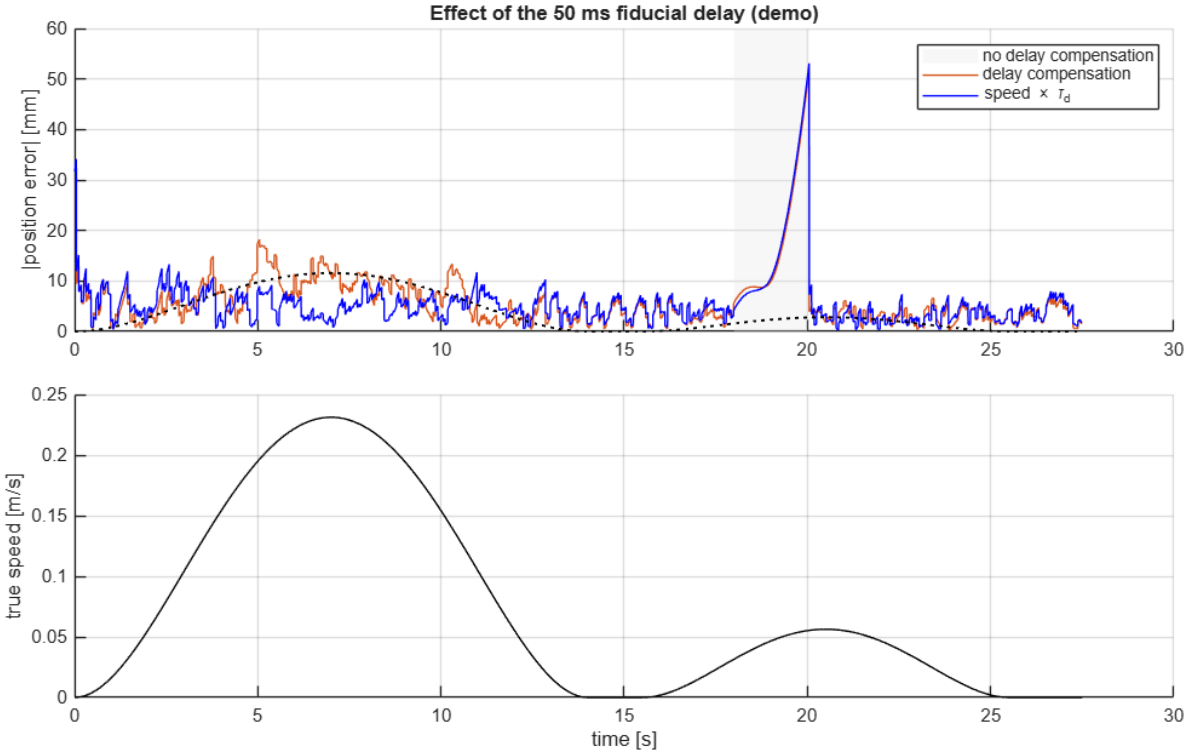


What it shows. As above, full scale.

How it was made. 20 runs.

What it supports. Full-scale mean 3.21, slightly below the band: the filter is conservative at the chosen q_v, kept for margin against the IMU-bias assumption.

`ekf_demo_delay.png`

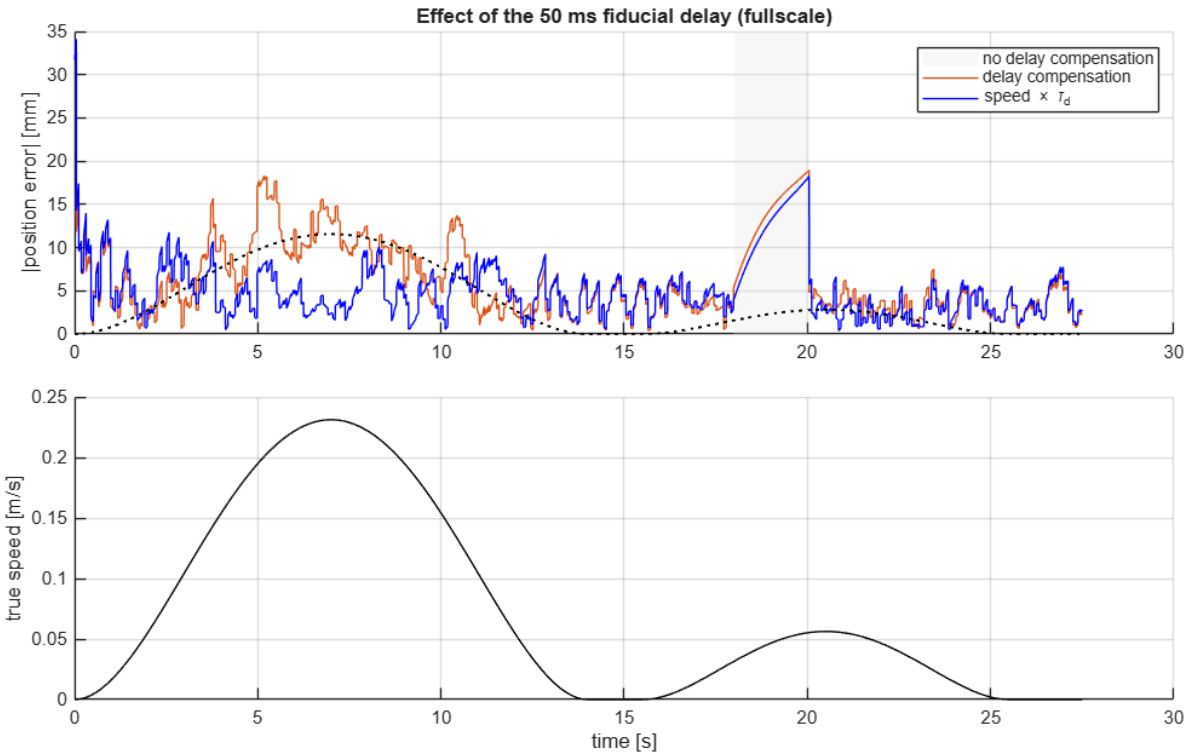


What it shows. Position-error magnitude during the constant-velocity cruise with delay compensation on and off, with the line speed$\times\tau_d$; true speed in the lower panel.

How it was made. Same sensor record processed twice, with and without the buffered innovation.

What it supports. Justifies the ring-buffer delayed innovation: 10 mm lag bias at 0.2 m/s without it, comparable to the demo tolerance; 1.6 mm with it.

`ekf_fullscale_delay.png`

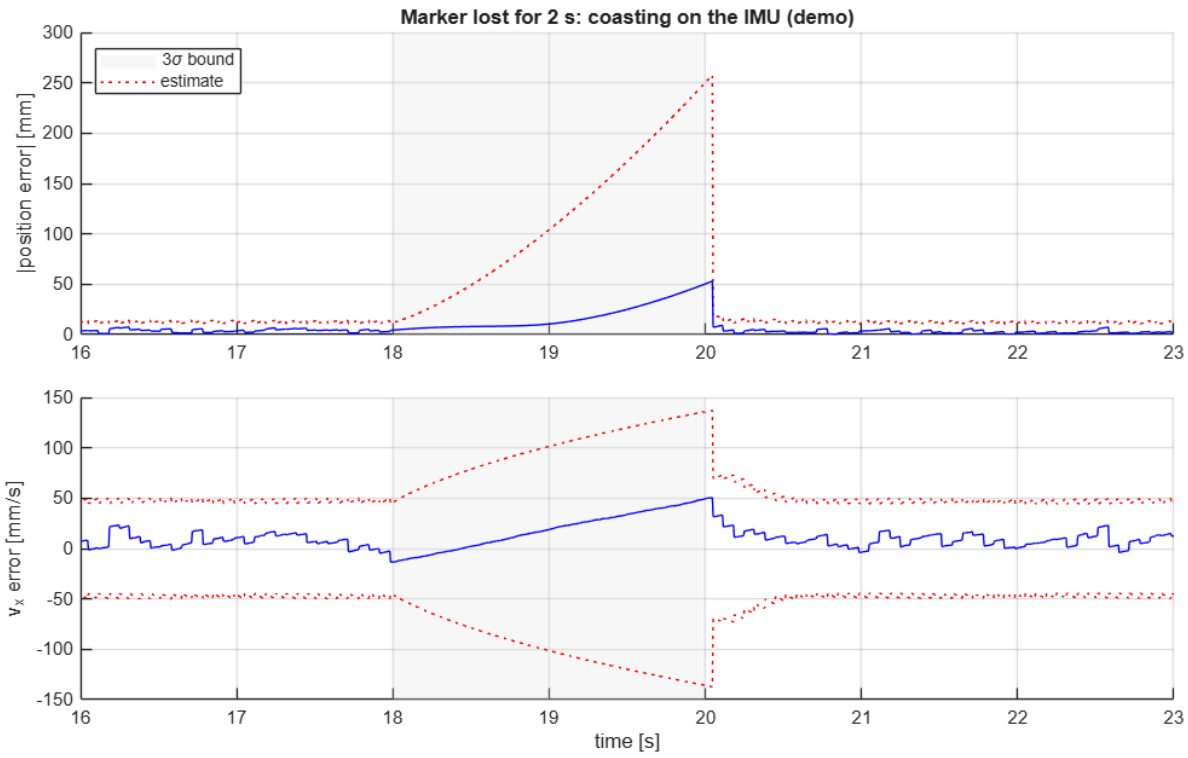


What it shows. As above, full scale.

How it was made. Same.

What it supports. Same conclusion at full scale.

`ekf_demo_dropout.png`



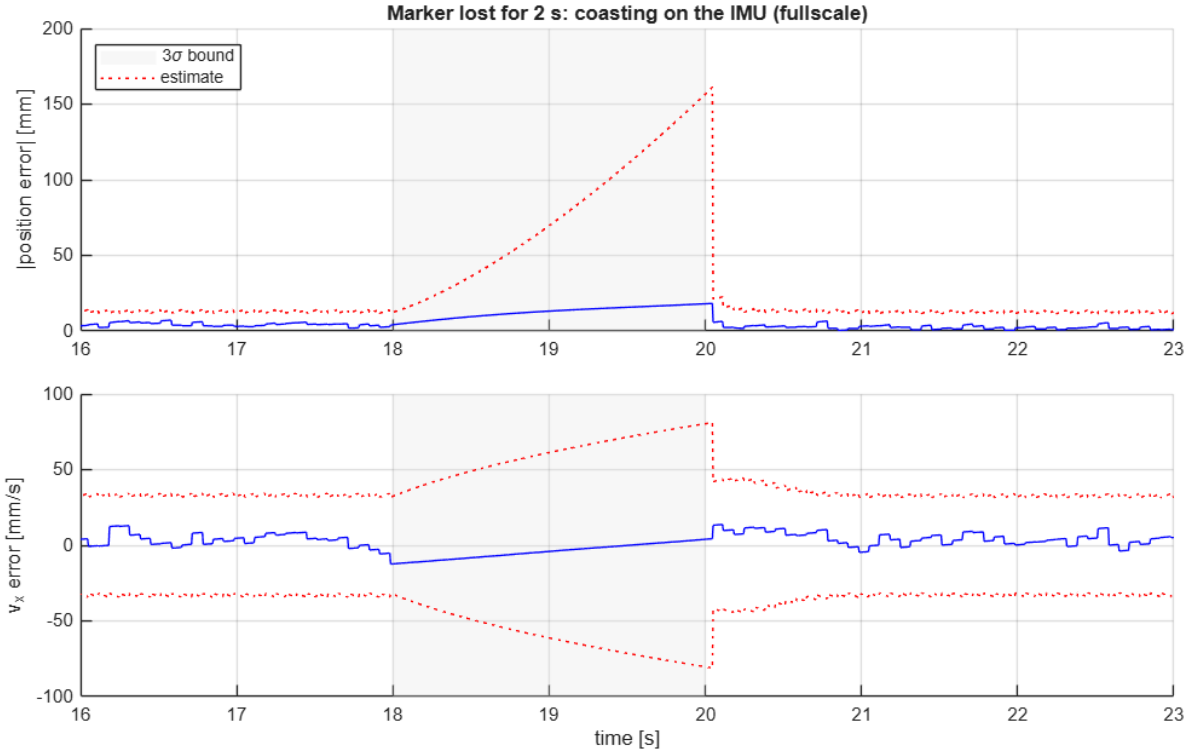
What it shows. Zoom on the 2 s dropout: position-error magnitude against the 3σ bound; v_x error

below.

How it was made. Same run as the errors figure.

What it supports. The demo leaves its 12 mm tolerance after about 0.2 s of coasting, so the demonstrator needs continuous marker visibility and the supervisor holds early.

`ekf_fullscale_dropout.png`

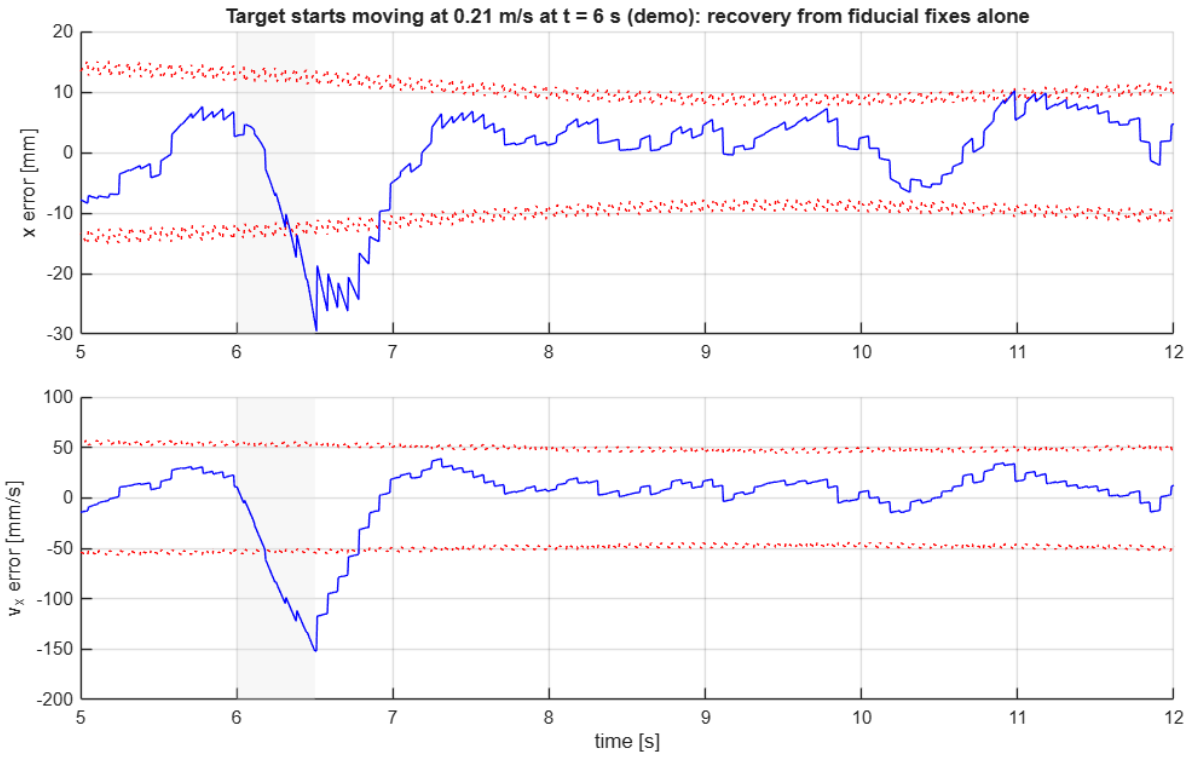


What it shows. As above, full scale.

How it was made. Same.

What it supports. Full scale stays inside the 80 mm tolerance for the whole 2 s, which sets the supervisor's abort time.

`ekf_demo_target_motion.png`

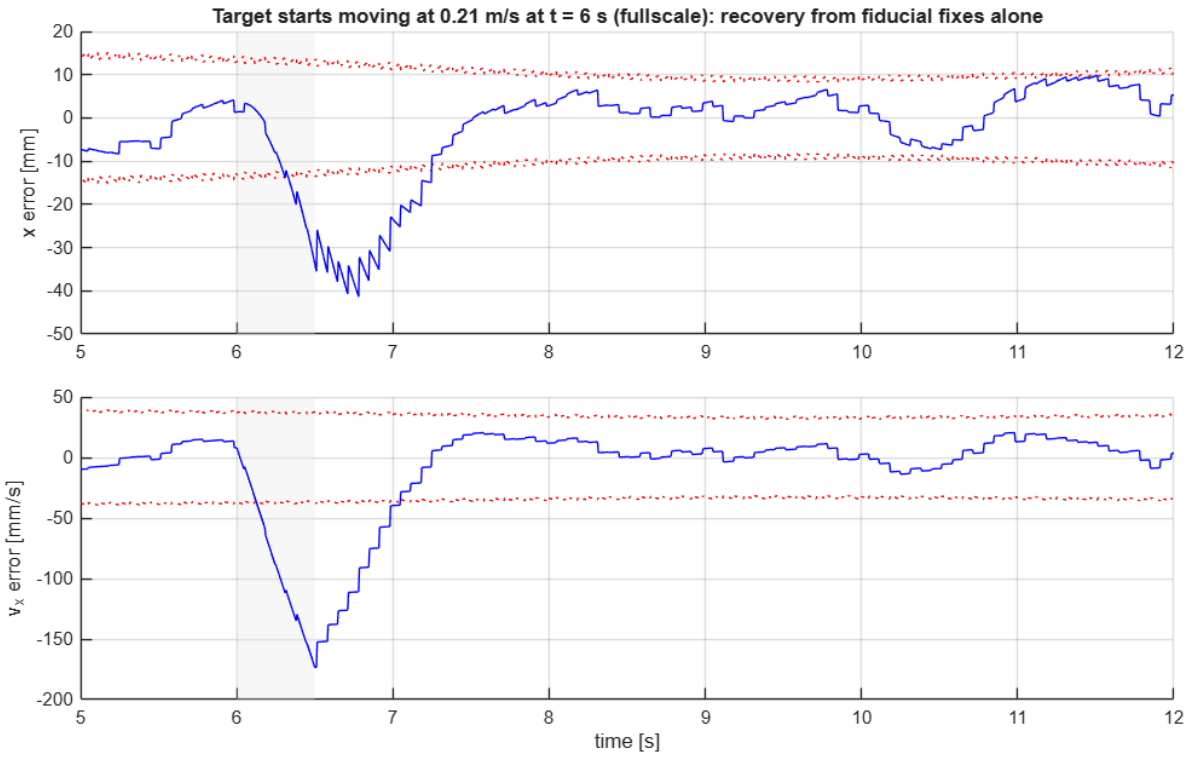


What it shows. The target starts moving at 0.2 m/s at t_{step}, which the robot's IMU cannot see: x error and v_x error over the next 6 s.

How it was made. `apply_target_velocity_step` on the truth trajectory, same sensors.

What it supports. The stationary-target assumption degrades gracefully: the filter re-converges from fixes alone within 1.2 s, no fix rejected. Also the evidence for the gate value: a tight gate rejected every fix here and the filter diverged.

`ekf_fullscale_target_motion.png`



What it shows. As above, full scale.

How it was made. Same.

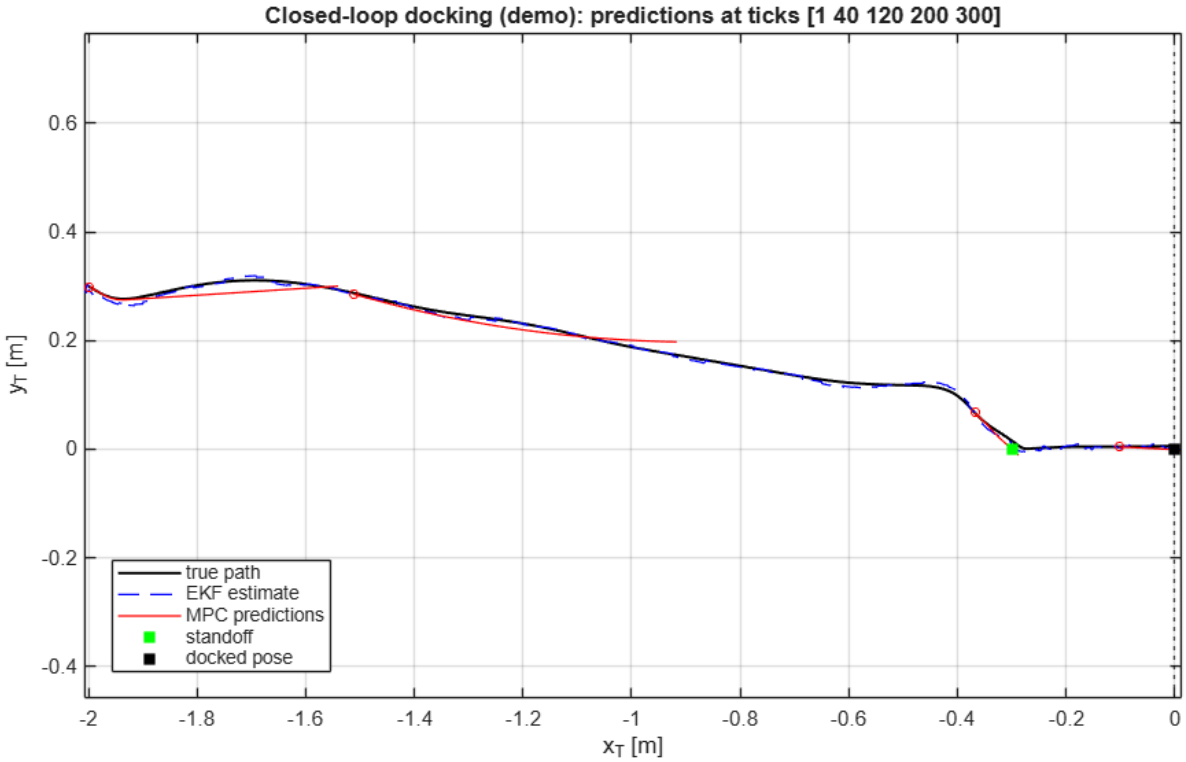
What it supports. Same conclusion.

---

2 Controller figures

Common setup: closed loop with the EKF in the loop from $(-2\text{ m}, 0.3\text{ m}, 15°)$, seed 1, truth plant with first-order wheel-speed loops (0.3 s full scale, 0.1 s demo) and a slip offset, three-stage supervisor.

`mpc_demo_plane.png`

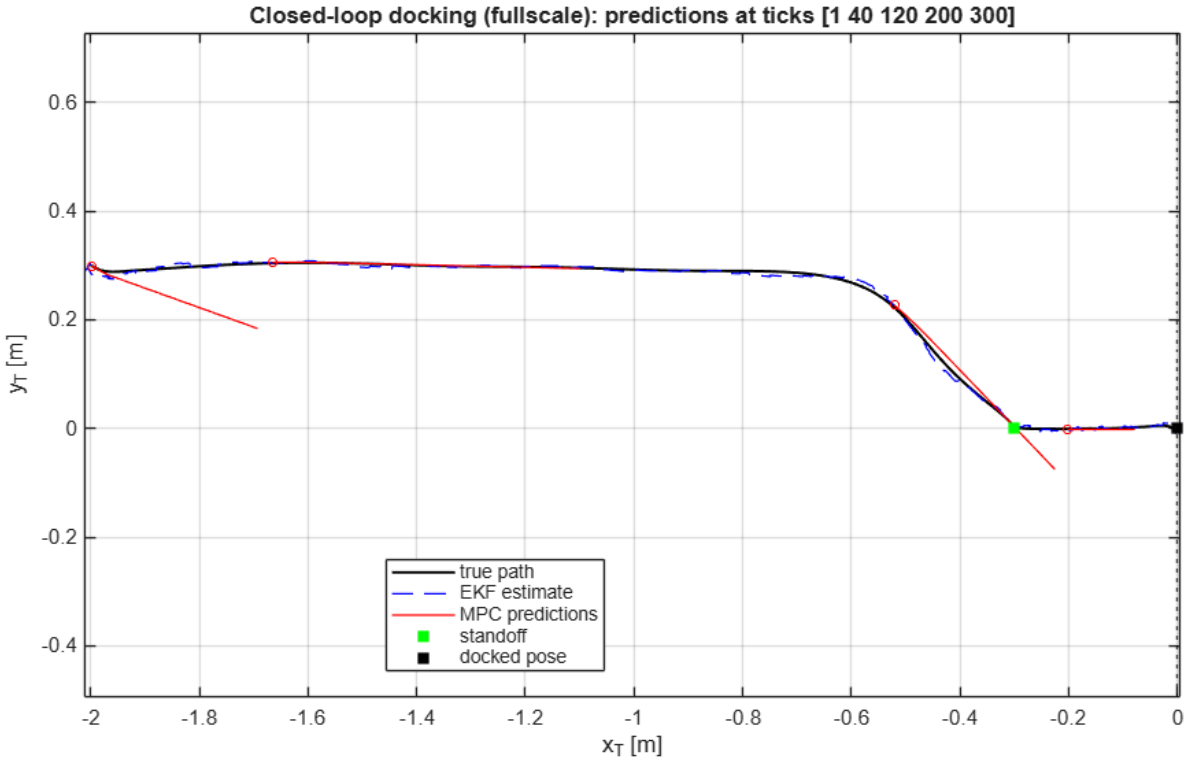


What it shows. Path in the target frame: true path, EKF estimate, and the MPC's predicted trajectories (red) at ticks 1, 40, 120, 200 and 300; standoff and docked markers.

How it was made. `make_mpc_figures`, demo.

What it supports. Receding horizon with successive linearisation: each red curve is the linear model's 2 s forecast from that tick; the forecasts curve as the linearisation point moves and converge onto the path.

`mpc_fullscale_plane.png`

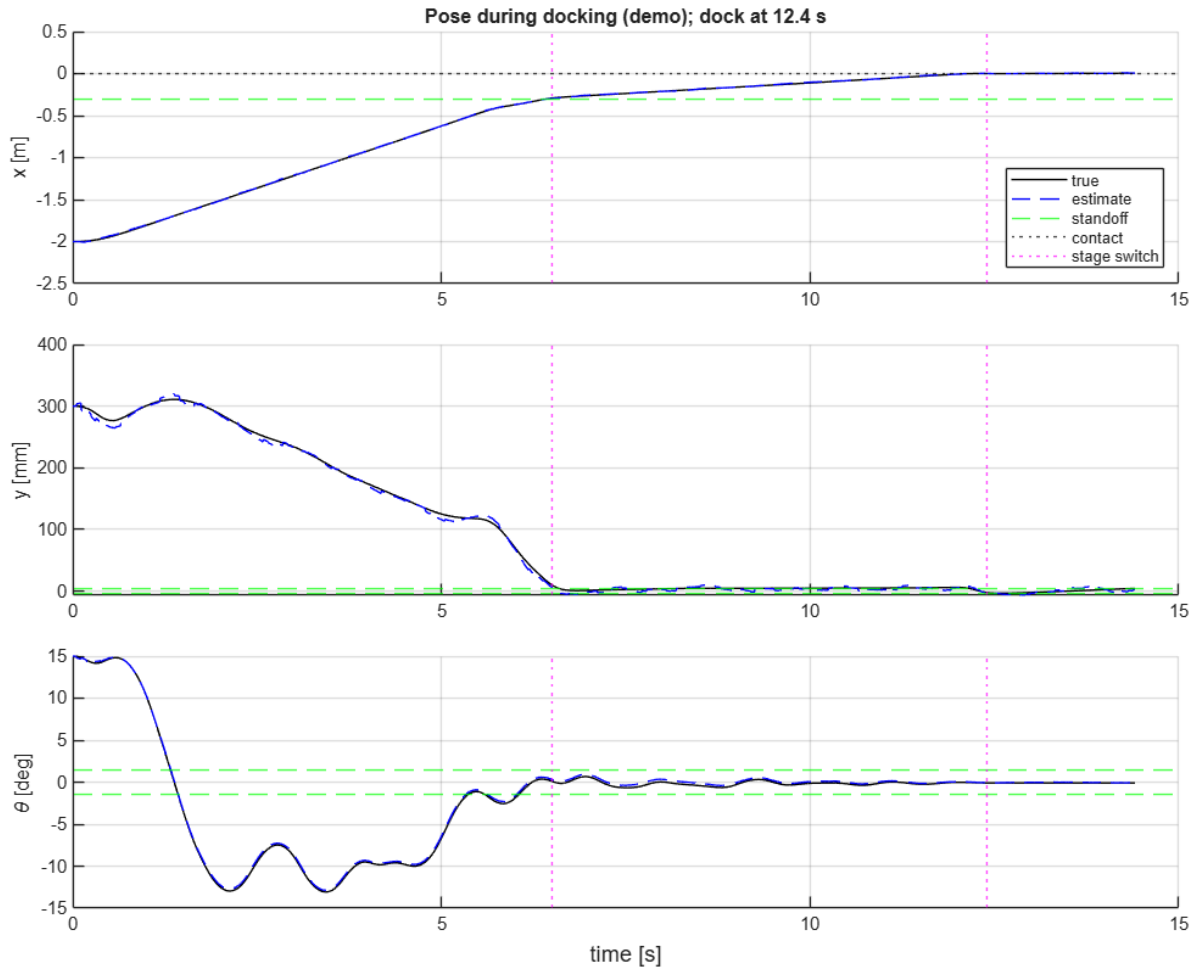


What it shows. As above, full scale.

How it was made. Same.

What it supports. Same role.

`mpc_demo_timeseries.png`

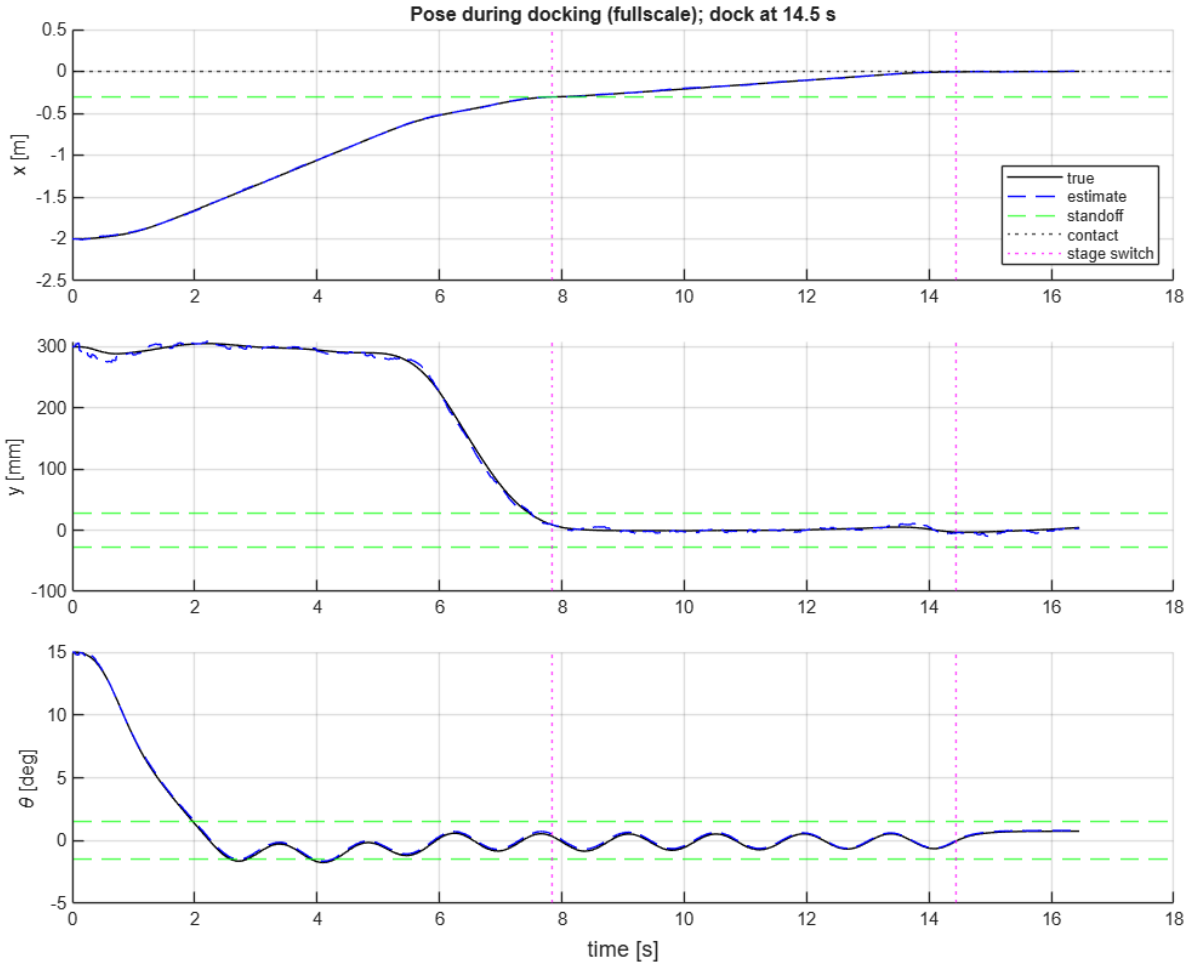


What it shows. x, y, θ against time, true and estimated, with the stage switches, the standoff and contact lines and the internal targets.

How it was made. Same run.

What it supports. The two-stage approach: alignment at the standoff, final approach at 50 mm/s, docking at 12.4 s. The heading settles within $\pm 1°$ after the initial turn.

`mpc_fullscale_timeseries.png`

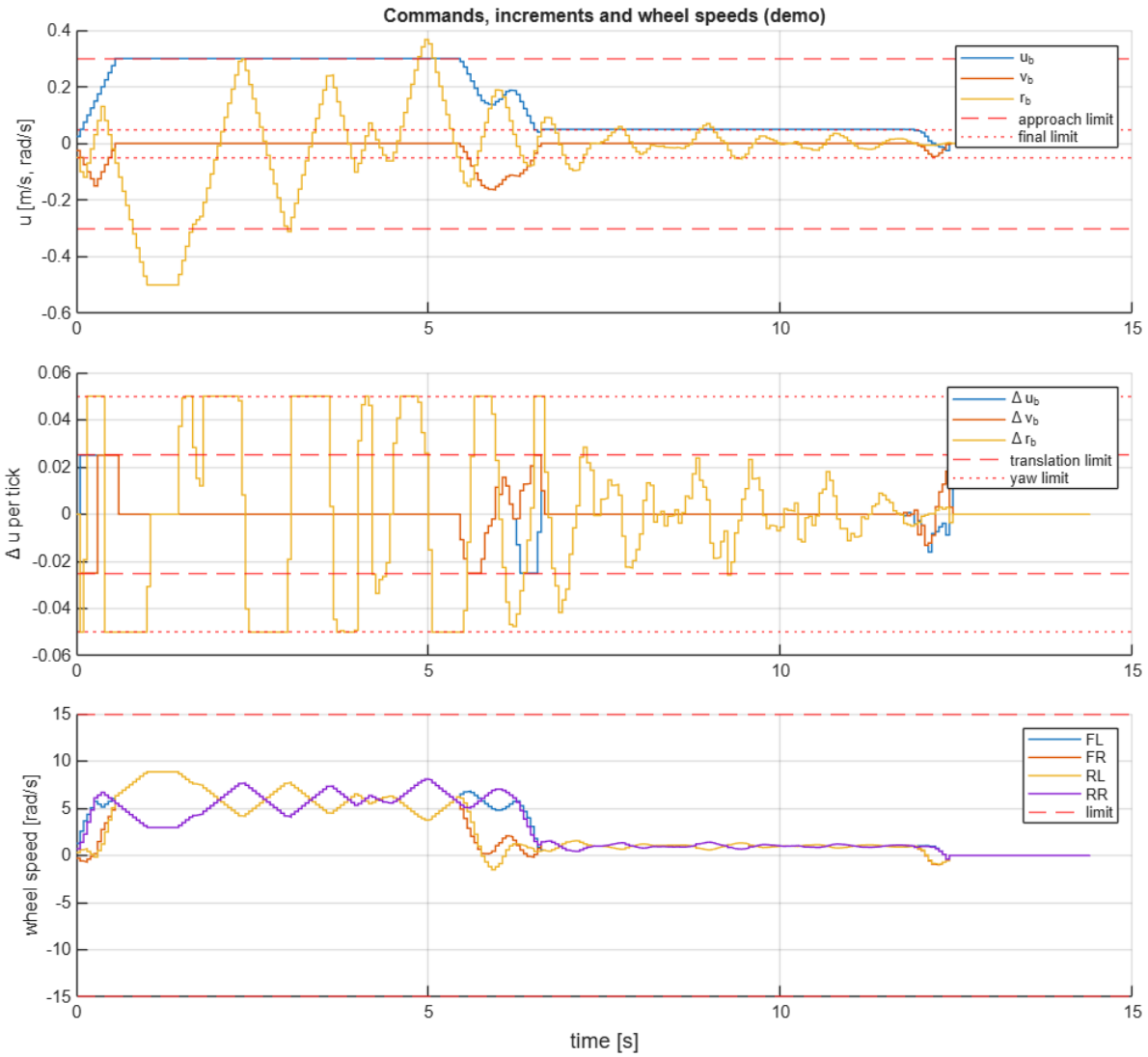


What it shows. As above, full scale; docking at 14.5 s.

How it was made. Same.

What it supports. The ±1° heading ripple is the yaw loop against the 0.3 s wheel lag the kinematic model omits; the lag model of the design document removes it.

`mpc_demo_commands.png`

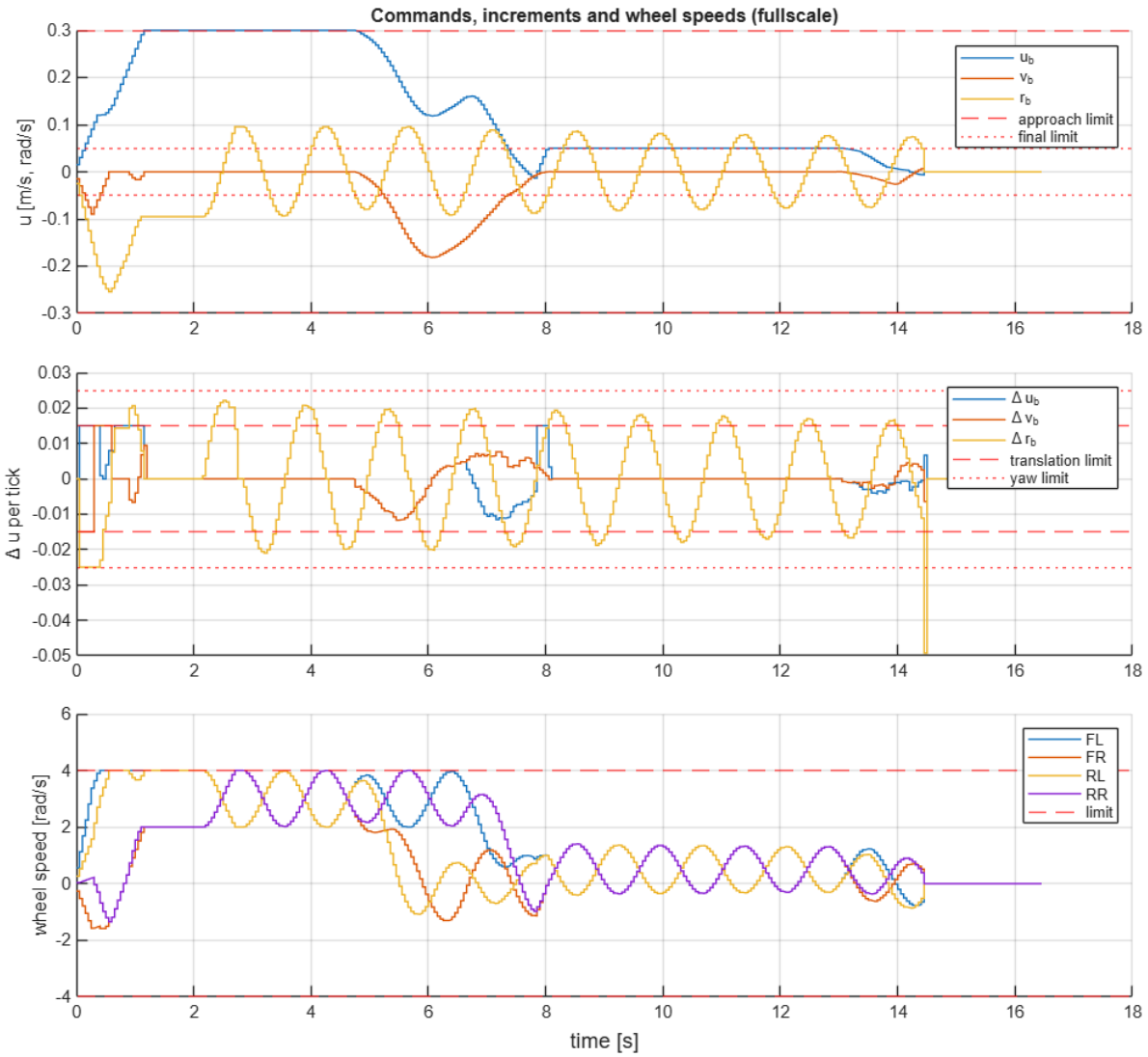


What it shows. Body-velocity commands with the approach and final caps; increments with the translation and yaw limits; the four wheel speeds with the wheel limit (the course's output / input / increment layout).

How it was made. Same run, sampled at the controller ticks.

What it supports. Which constraints are active when: the increment limit during the initial turn and the deceleration at the standoff; the L1 speed cap during the approach. The wheel limit is never exceeded.

`mpc_fullscale_commands.png`

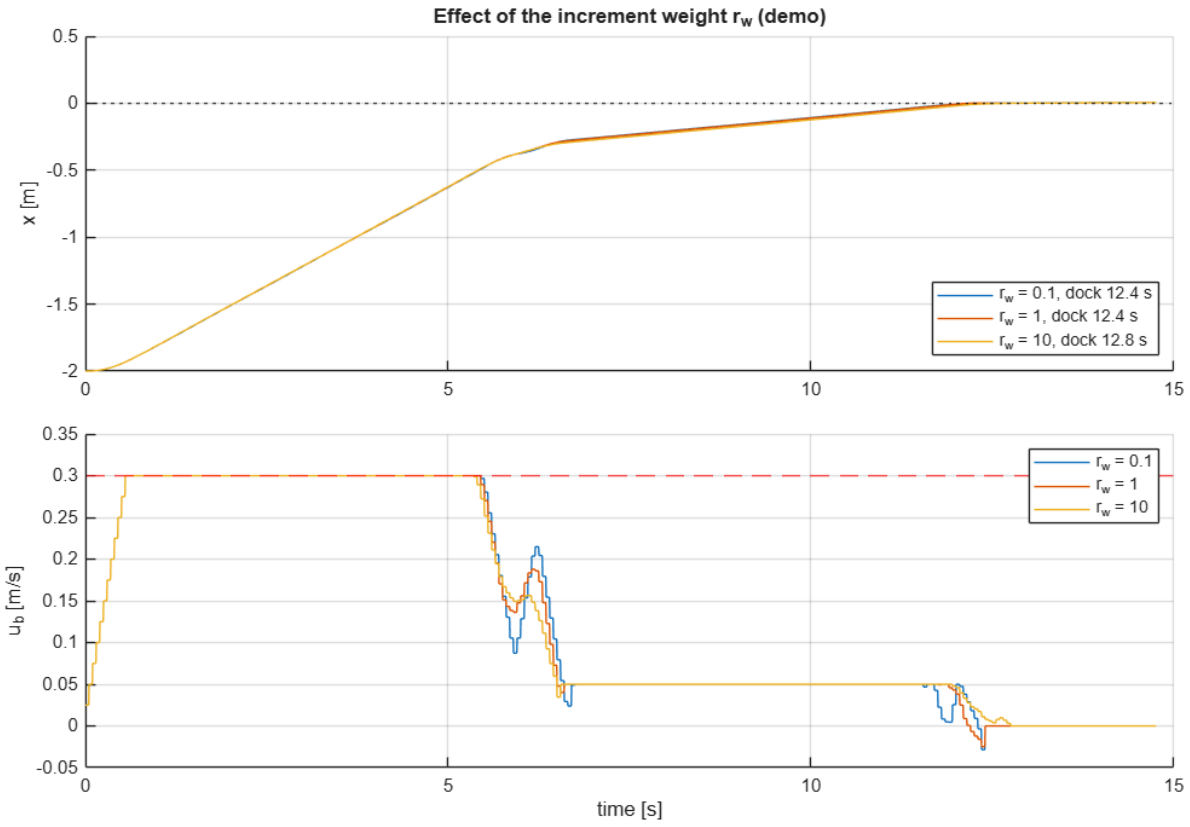


What it shows. As above, full scale.

How it was made. Same.

What it supports. At full scale the wheel limit (4 rad/s) is the active constraint during the approach, which is where the inverse kinematics inside the controller matters.

`mpc_demo_rw.png`

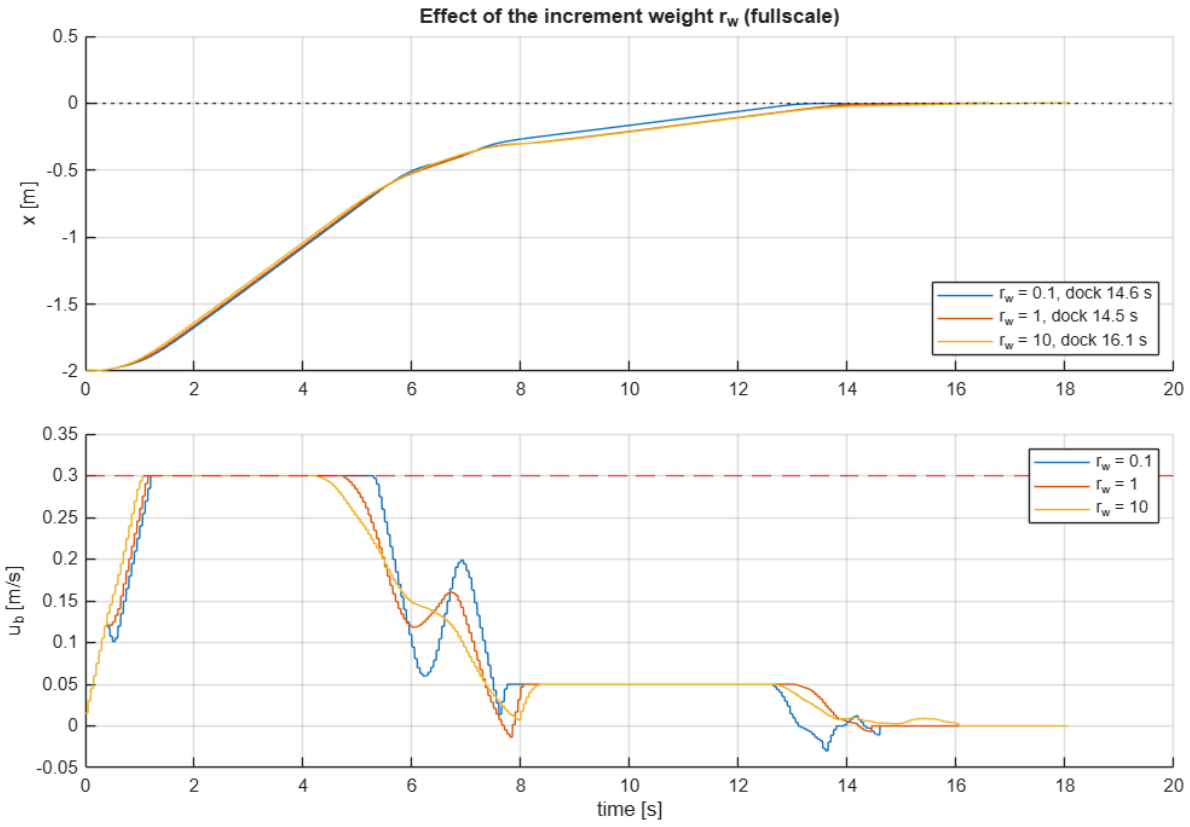


What it shows. $x(t)$ and the forward command for $r_w = 0.1$, 1 and 10, with the docking times.

How it was made. Three closed-loop runs, same seed.

What it supports. The course's single tuning knob: smaller r_w gives more aggressive increments; the design uses $r_w = 1$.

`mpc_fullscale_rw.png`

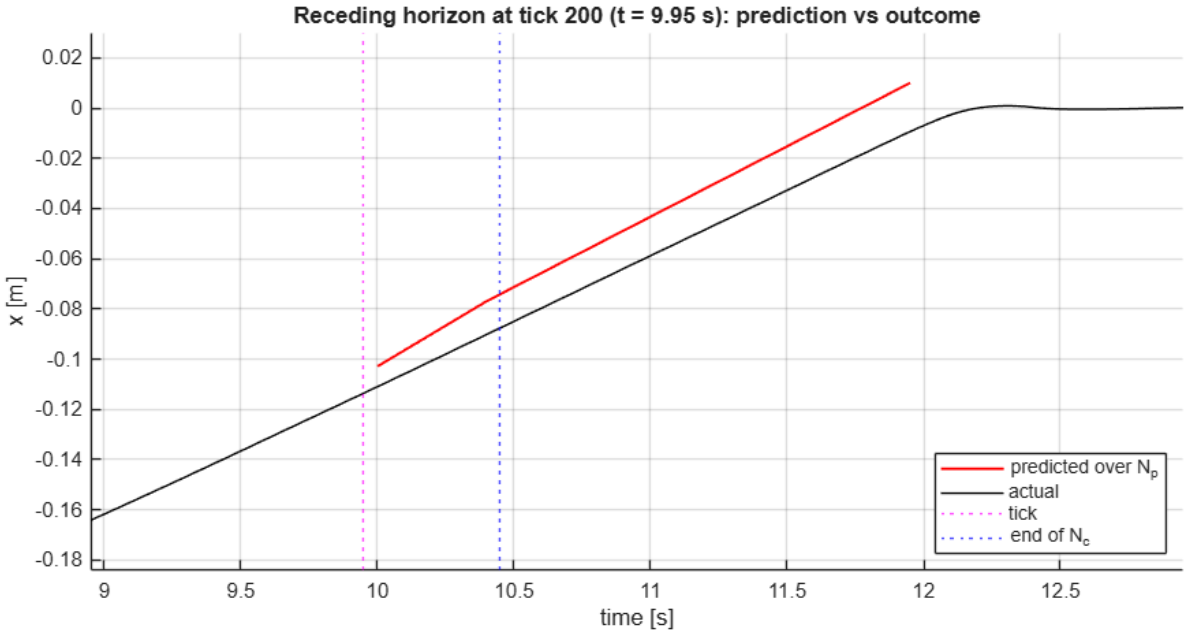


What it shows. As above, full scale.

How it was made. Same.

What it supports. Same role.

`mpc_demo_tick.png`

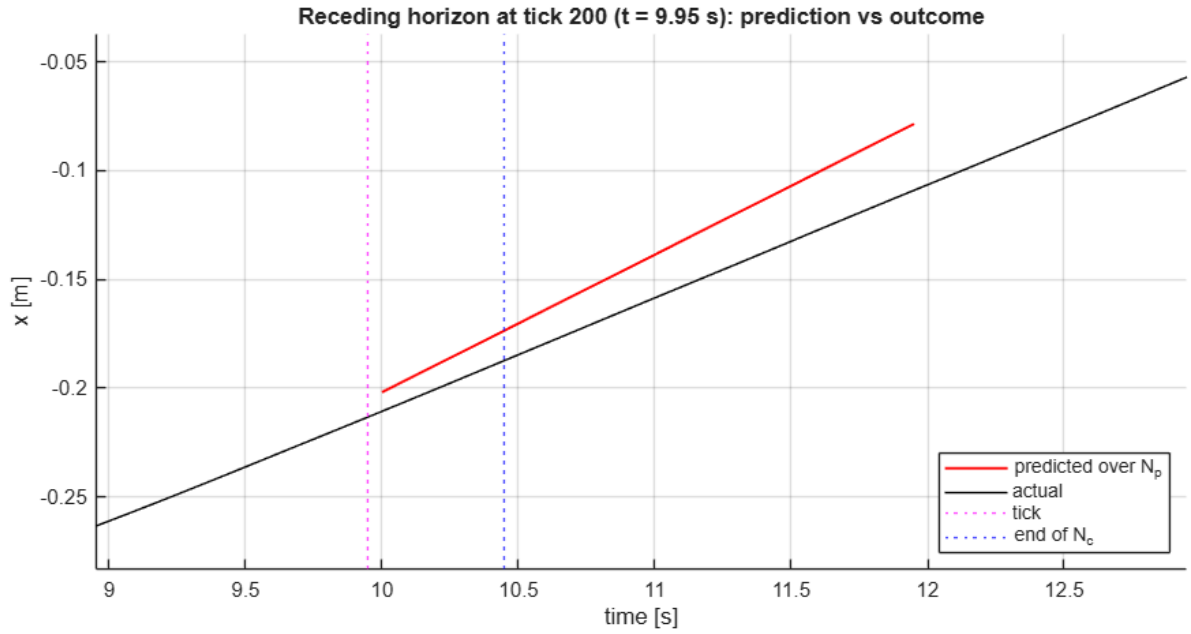


What it shows. One tick's predicted x over the 2 s horizon (red) against what actually happened (black), with the tick and the end of the control horizon marked.

How it was made. Prediction stored at tick 200 of the nominal run.

What it supports. Prediction quality of the per-tick linear model, the Example 5.3 flavour of the notes.

`mpc_fullscale_tick.png`



What it shows. As above, full scale.

How it was made. Same.

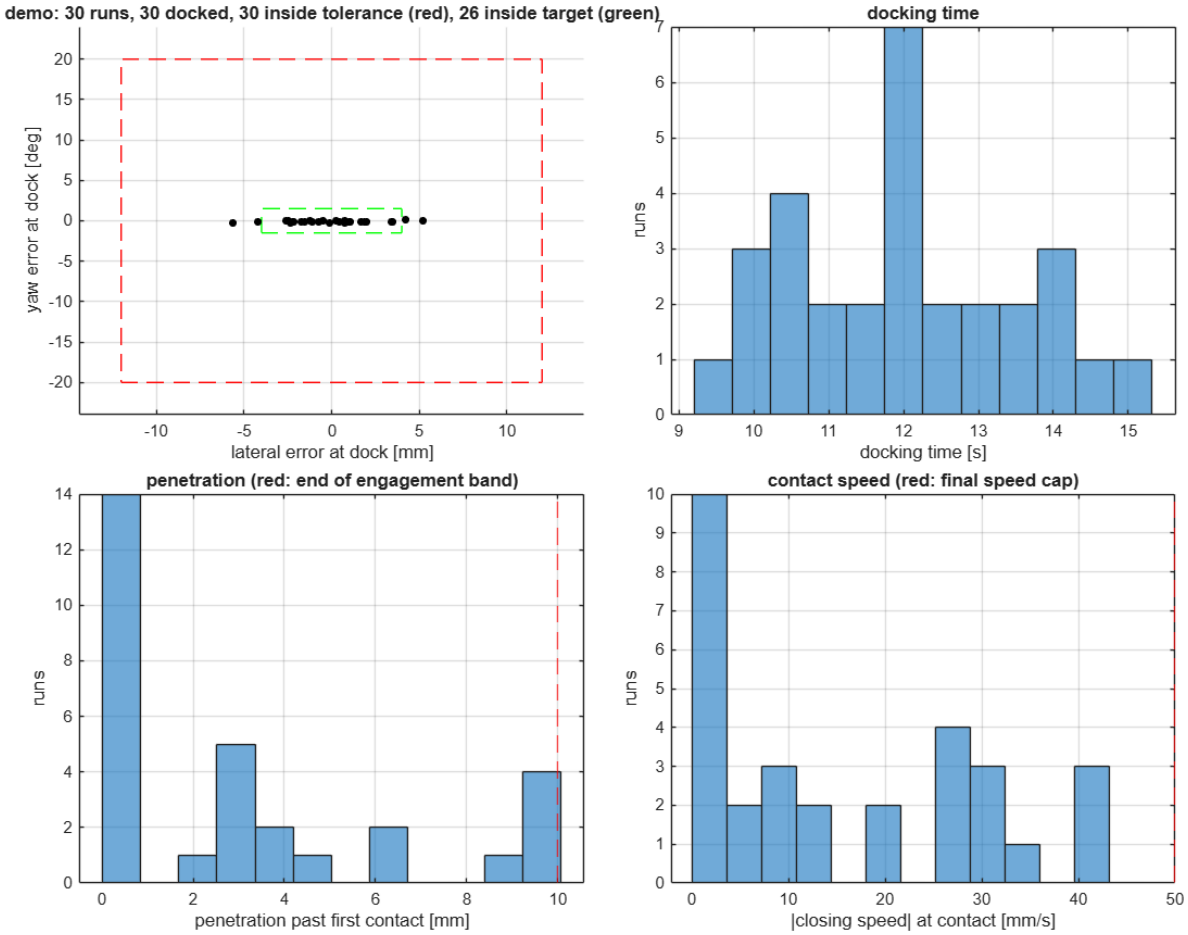
What it supports. Same role.

---

3 Report figures

All closed loop with the EKF in the loop, produced by `make_report_figures`.

`report_demo_montecarlo.png`

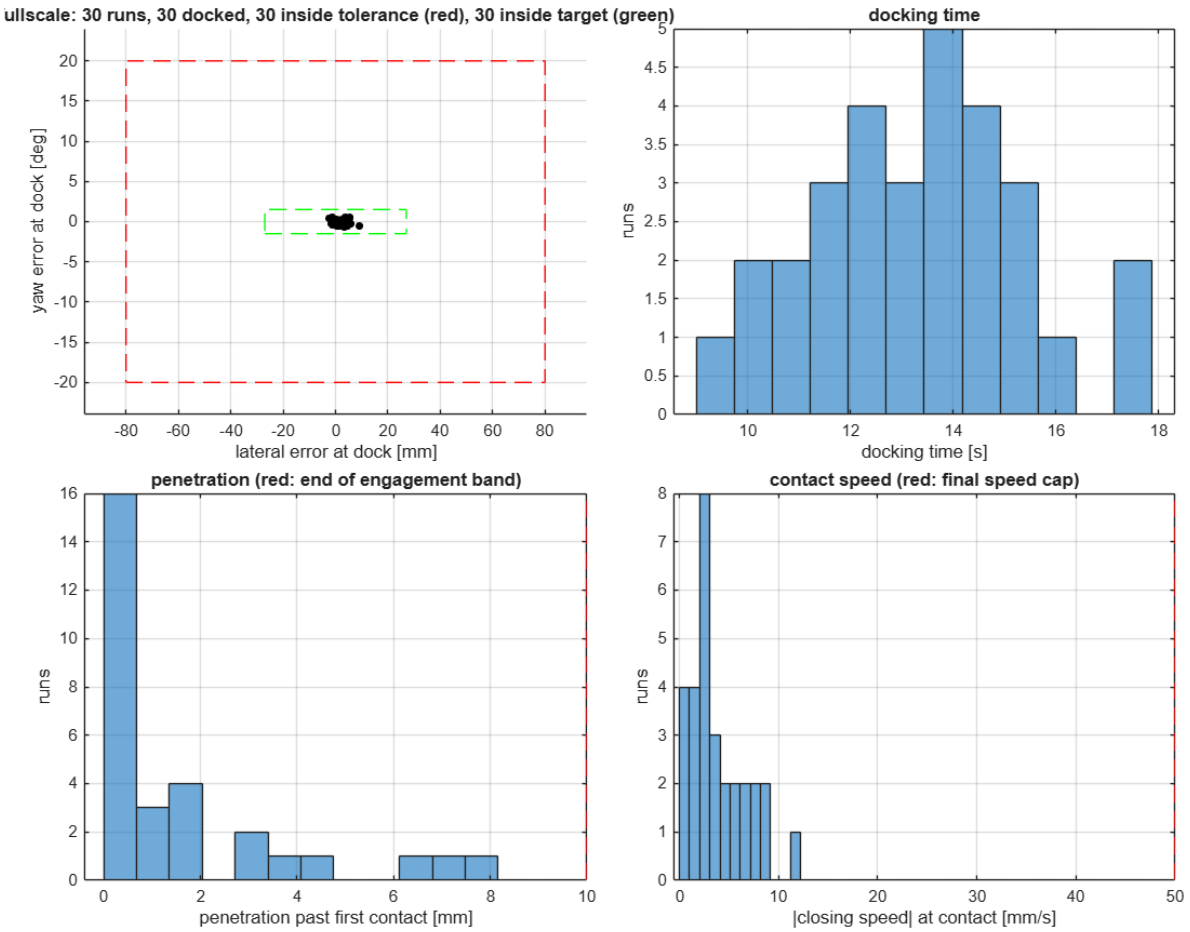


What it shows. 30 runs from start poses drawn uniformly in $x \in [-2.5, -0.8]$ m, $y \in [-0.4, 0.4]$ m, $\theta \in [-20°, 20°]$ with independent noise seeds. Top left: final lateral and yaw error against the coupler tolerance (red box) and the internal target (green box); then docking time, penetration past first contact (red line: end of the 10 mm engagement band) and closing speed at contact (red line: final cap).

How it was made. `make_report_figures`, demo.

What it supports. The main repeatability result for the demo: 30/30 inside the tolerance, 26/30 inside the 4 mm target; lateral 2.4 mm rms, 5.6 mm max; yaw $\leq 0.26°$; docking 12.0 s mean, 15.2 s max; penetration ≤ 10.1 mm; contact ≤ 43 mm/s; no solver fallback. The four runs outside the target are estimator-limited: the assumed 4 mm fiducial noise per fix equals the target.

`report_fullscale_montecarlo.png`

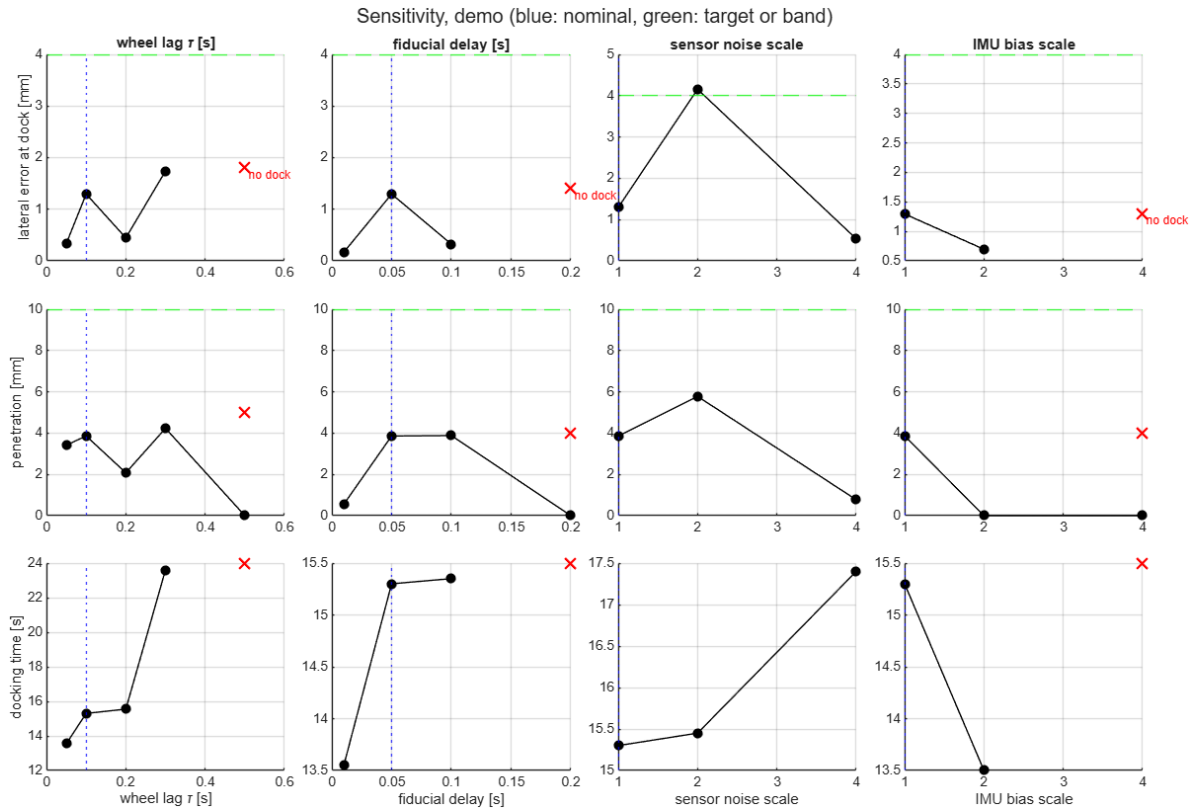


What it shows. As above, full scale.

How it was made. Same.

What it supports. 30/30 inside tolerance and target; lateral 3.5 mm rms, 9.2 mm max against the 27 mm target; yaw $\leq 0.61°$; docking 13.3 s mean, 17.9 s max; penetration ≤ 8.2 mm; contact ≤ 12 mm/s.

`report_demo_sensitivity.png`

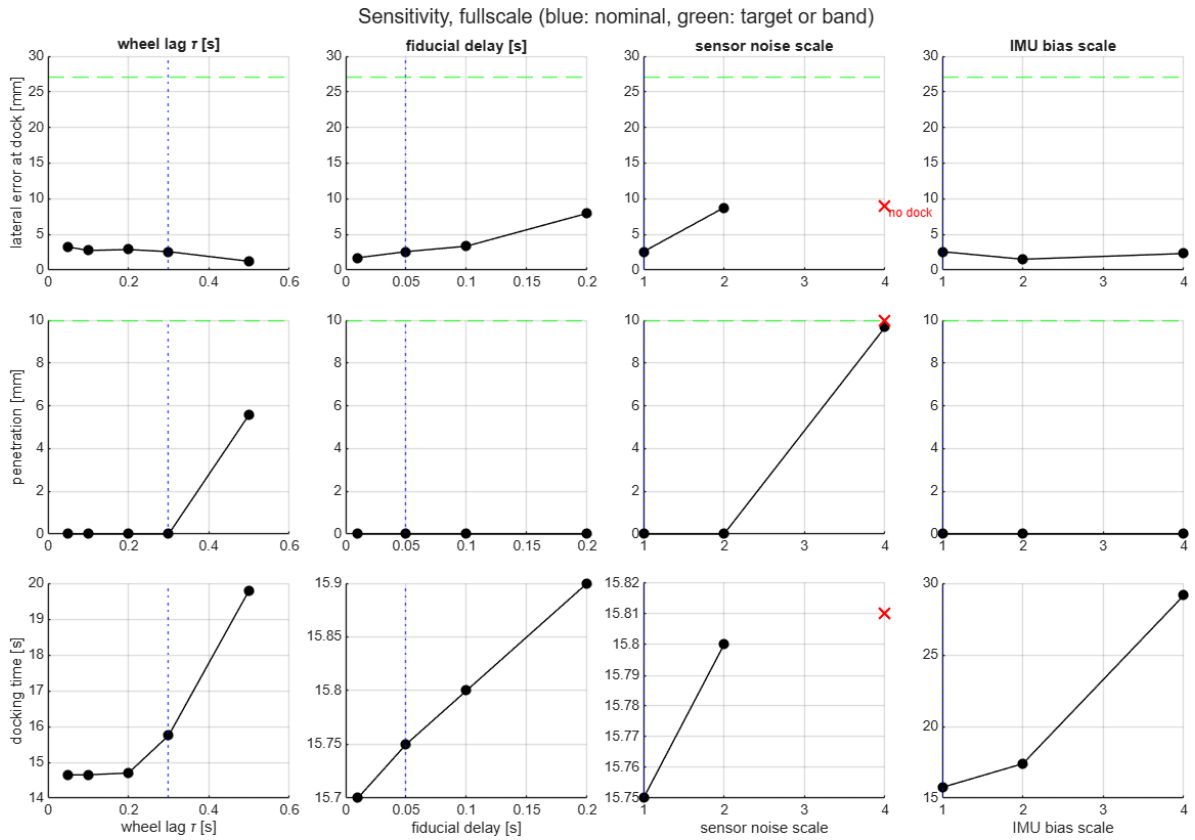


What it shows. 3×4 grid: lateral error at docking, penetration and docking time against the wheel-loop time constant τ, the fiducial delay, the sensor-noise scale and the IMU-bias scale, one at a time, with the filter and controller unaware of the change. Blue dotted = nominal, green = target or band, red $\times$ = did not dock within 40 s.

How it was made. 15 closed-loop runs from the nominal start pose, seed 21.

What it supports. The demo's margins: docks with τ up to 0.3 s, delay up to 0.1 s, noise up to $4\times$, bias up to $2\times$; does not dock at $\tau = 0.5$ s, delay 0.2 s or bias $4\times$. Hence the bench requirements: $\tau < 0.2$ s, latency < 0.1 s, residual bias within $2\times$ of 0.03 m/s^2.

`report_fullscale_sensitivity.png`

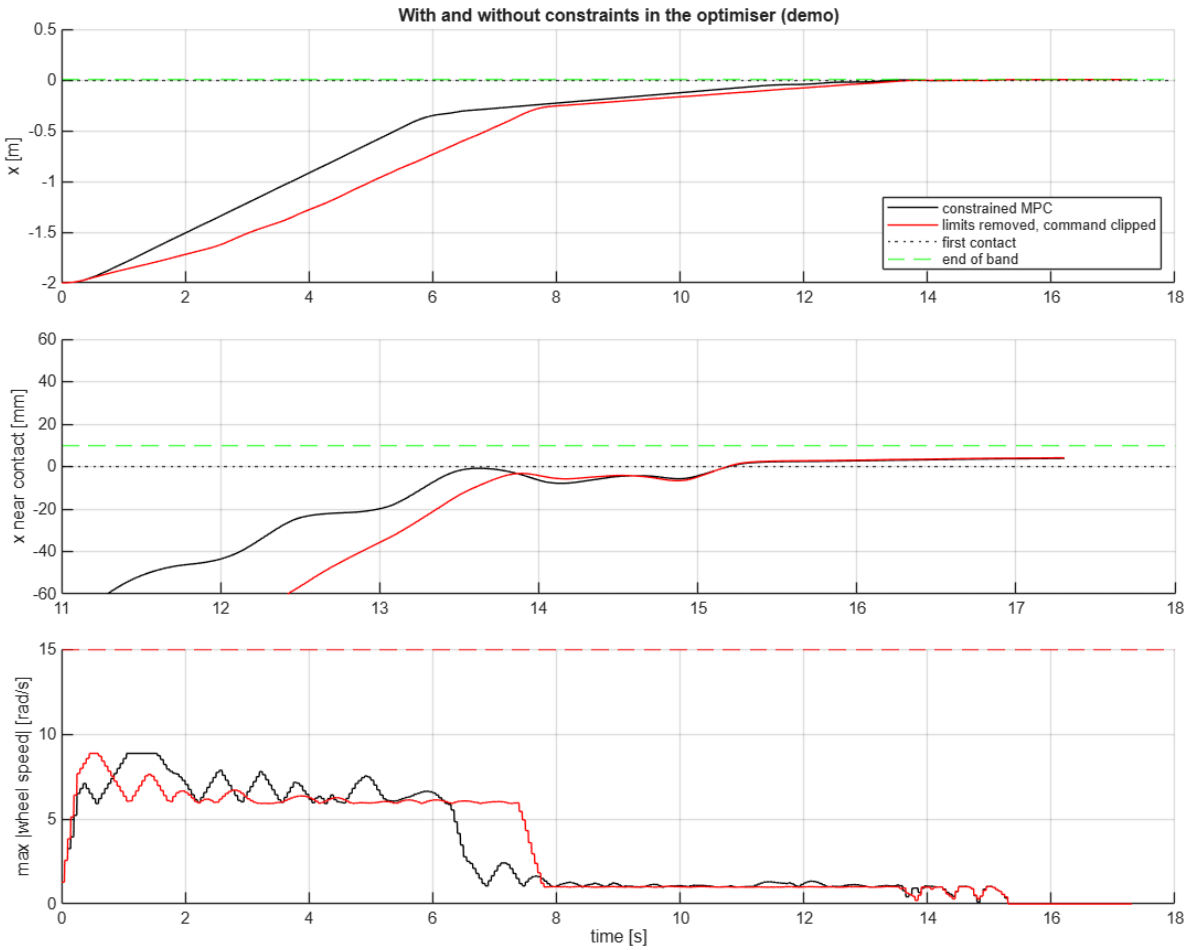


What it shows. As above, full scale.

How it was made. Same.

What it supports. Full scale is insensitive over the ranges tested: τ up to 0.5 s, delay up to 0.2 s and bias up to 4× all dock inside the targets; only noise at 4× prevents the alignment condition being met within 40 s.

`report_demo_constraints.png`

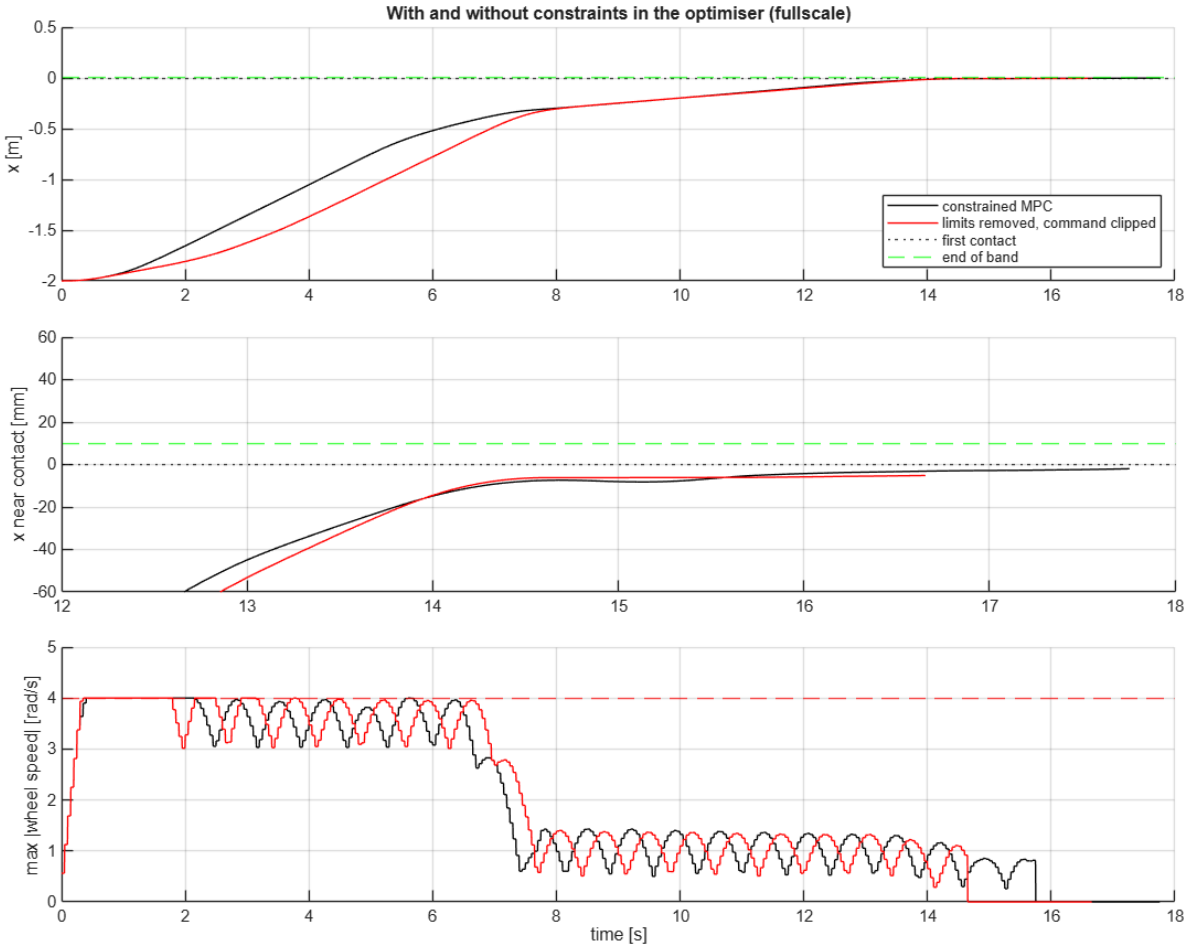


What it shows. The nominal run with the constraints in the optimiser (black) against the same controller with every limit removed from the QP and the command clipped to the limits afterwards (red): $x(t)$, x near the contact plane, largest wheel speed.

How it was made. Two closed-loop runs, same seed; the clipped version uses the `unconstrained` option of `run_closed_loop`.

What it supports. An honest comparison: in the nominal run the two behave alike (penetration 3.9 against 4.2 mm) because the active limits are enforced equally by clipping and the output constraints are inactive. The constraints matter for the output rows, which clipping cannot enforce, and for the feasibility guarantee; the nominal performance does not rest on them.

`report_fullscale_constraints.png`

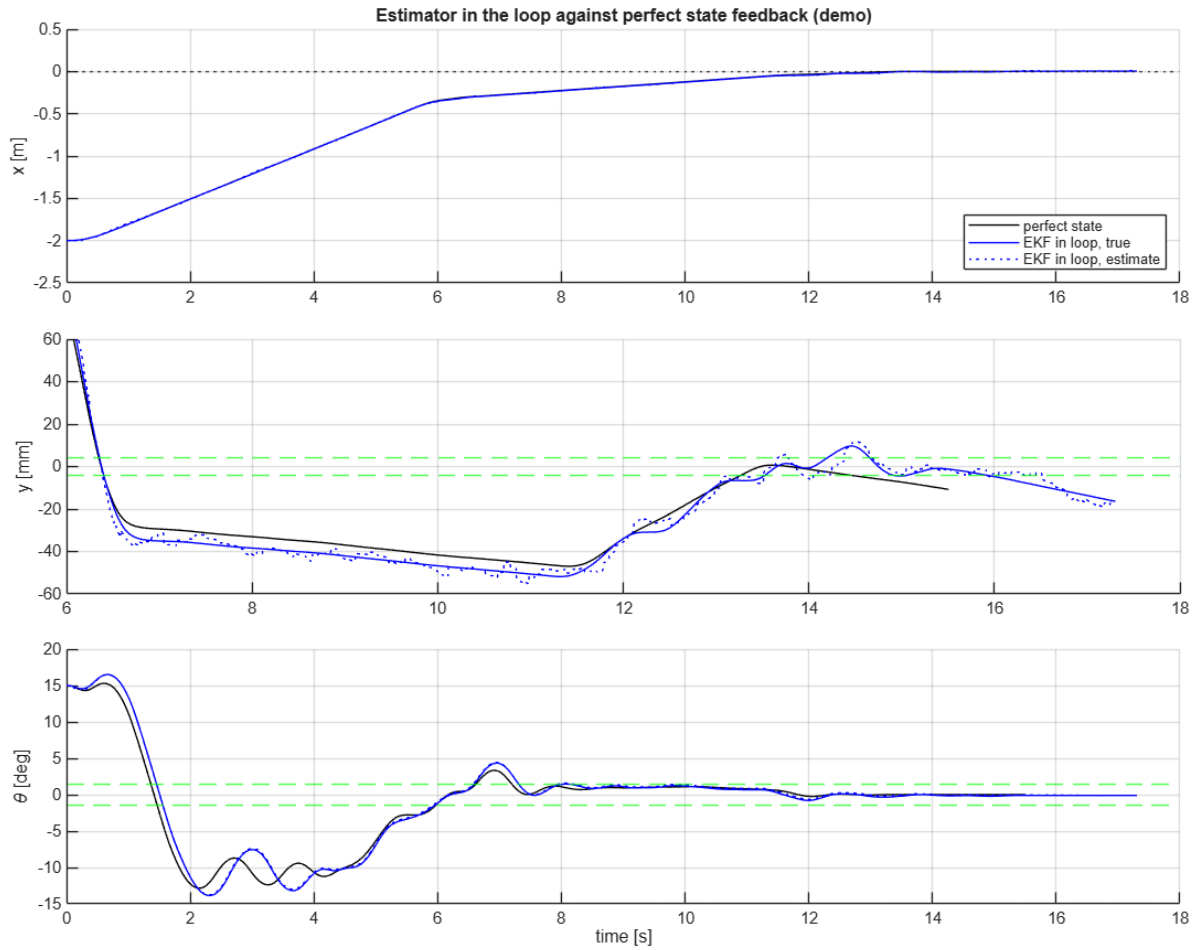


What it shows. As above, full scale.

How it was made. Same.

What it supports. No penetration in either case; the clipped run is in fact 1 s faster. Same conclusion.

`report_demo_estimator.png`

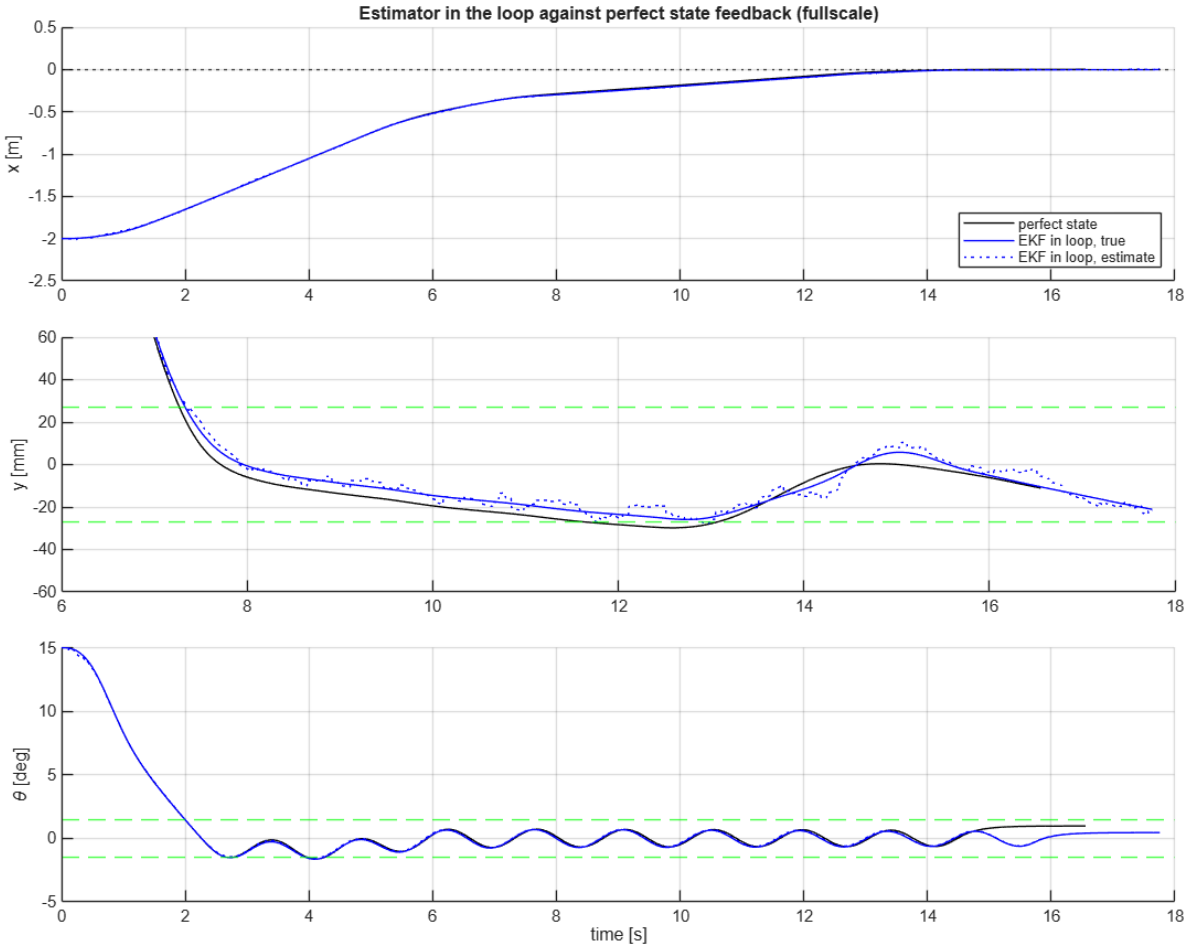


What it shows. Perfect state feedback (black) against the EKF in the loop, true pose (blue) and estimate (dotted): x, y, θ.

How it was made. Two closed-loop runs, same seed, `use_ekf` on and off.

What it supports. The estimator's cost: 15.3 against 13.5 s docking time and 1.3 against 0.2 mm final lateral error; the estimate stays within its covariance throughout. Certainty equivalence holds for this loop.

`report_fullscale_estimator.png`



What it shows. As above, full scale.

How it was made. Same.

What it supports. 15.8 against 14.6 s and 2.6 against 0.6 mm.
